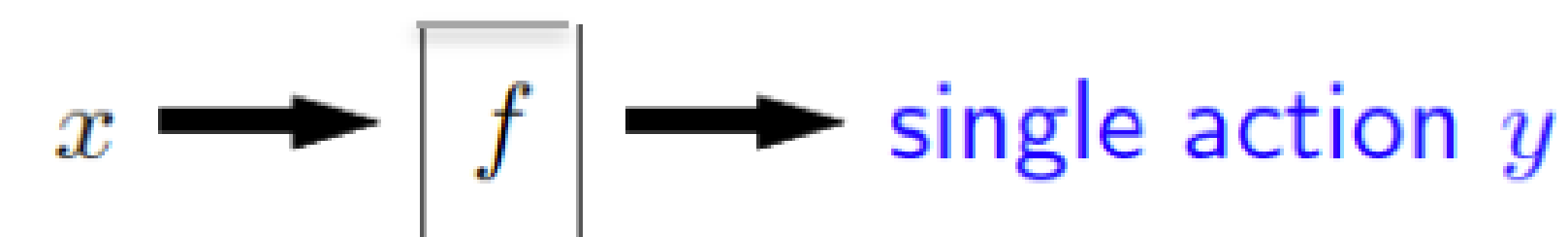


Recap: From reflex to problem solving agents

• Reflex Agents

- Directly map states to actions.
- No notion of goals. Do not consider action consequences.
- Operate using **condition-action** rules:
If (condition) **then** (action)



• Problem Solving Agents

- Adopt a goal
- Plan ahead: Consider future actions and the desirability of their outcomes
- Solution: a **sequence of actions** that the agent can execute leading to the goal.



Search is the method that enables such agents to find that sequence.

Reference: Stuart Russell & Peter Norvig, *Artificial Intelligence: A Modern Approach* (4th ed.) P81-P82

Problem solving as search

Problem-Solving Agents

- Adopt a goal to achieve
- Plan ahead by considering future actions and the desirability of their outcomes
- There is a sequence of actions the agent can execute to reach the goal

Search

- *Generic definition:* The computational process that a problem-solving agent uses to find that optimal sequence of actions.
- *Narrow definition:* The specific algorithm used to find an optimal path to the goal.

Reference: Stuart Russell & Peter Norvig, *Artificial Intelligence: A Modern Approach* (4th ed.) P81-P82

Formulating search problem: 8-puzzle

Formulate **goal**

- Pieces to end up in order as shown

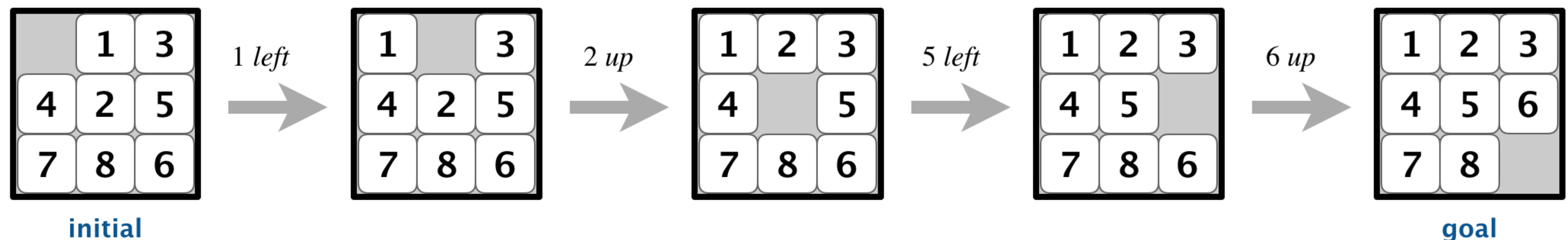
The process of
formulate a path

Formulate **search problem**

- **States**: configurations of the puzzle
- **Actions**: Move one of the movable pieces
- **Performance** measure: minimize total moves

Find **solution**

- Sequence of pieces moved: 1, 2, 5, 6



Reference: Stuart Russell & Peter Norvig, *Artificial Intelligence: A Modern Approach* (4th ed.) P83 – P88.

General search problems formulation

In general, a search problem is defined by:

1. **State space**: a set S containing all possible states of the environment.
2. **Initial state**: $s_i \in S$, indicating where the agent starts.
3. **Actions**: a set A containing possible moves from a state.
 $\forall s \in S: A = \text{ACTIONS}(s) = \text{actions executable in } s$
4. **Transition model**: Result of applying an action.
 $\forall a \in \text{ACTIONS}(s), \text{RESULT}(s, a) \rightarrow s', s'$ is called a successor of s
At state s , how many actions can be taken
5. **Action cost**: Cost of applying an action, $\text{ACTION-COST}(s, a, s')$, aligned with the performance measure
6. **Goal state(s)**: Desired state(s) with a desired property
The agent uses a **goal test** to check if the current state is a goal state
 $\text{IS-GOAL}(s)$: s is a goal state if $\text{IS-GOAL}(s)$ is true

Reference: Stuart Russell & Peter Norvig, *Artificial Intelligence: A Modern Approach* (4th ed.) P83

Key concepts: Agent and state

Agent (simplified definition):

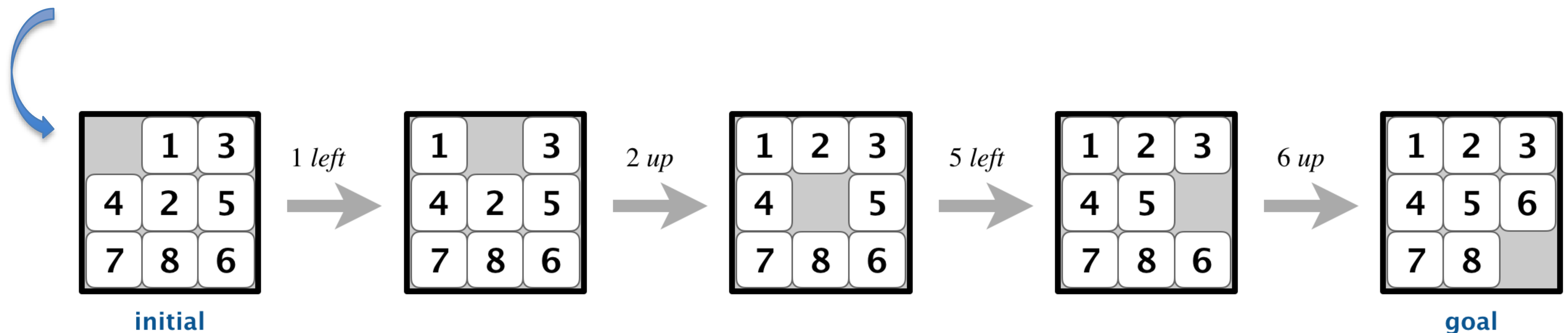
entity that perceives its environment and acts upon that environment

State:

a configuration of the agent and its environment, typically represented by the letter s .

Initial state:

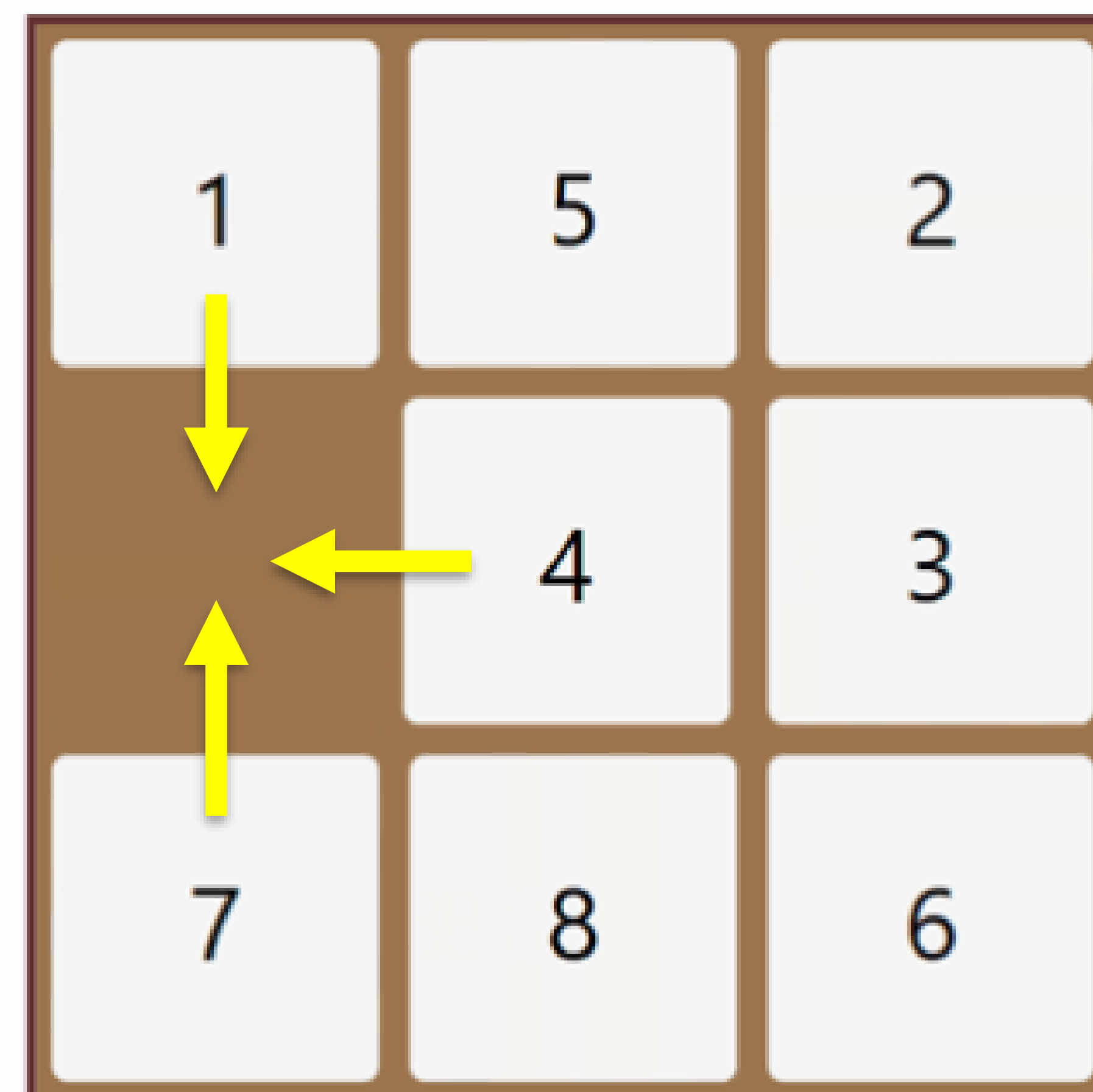
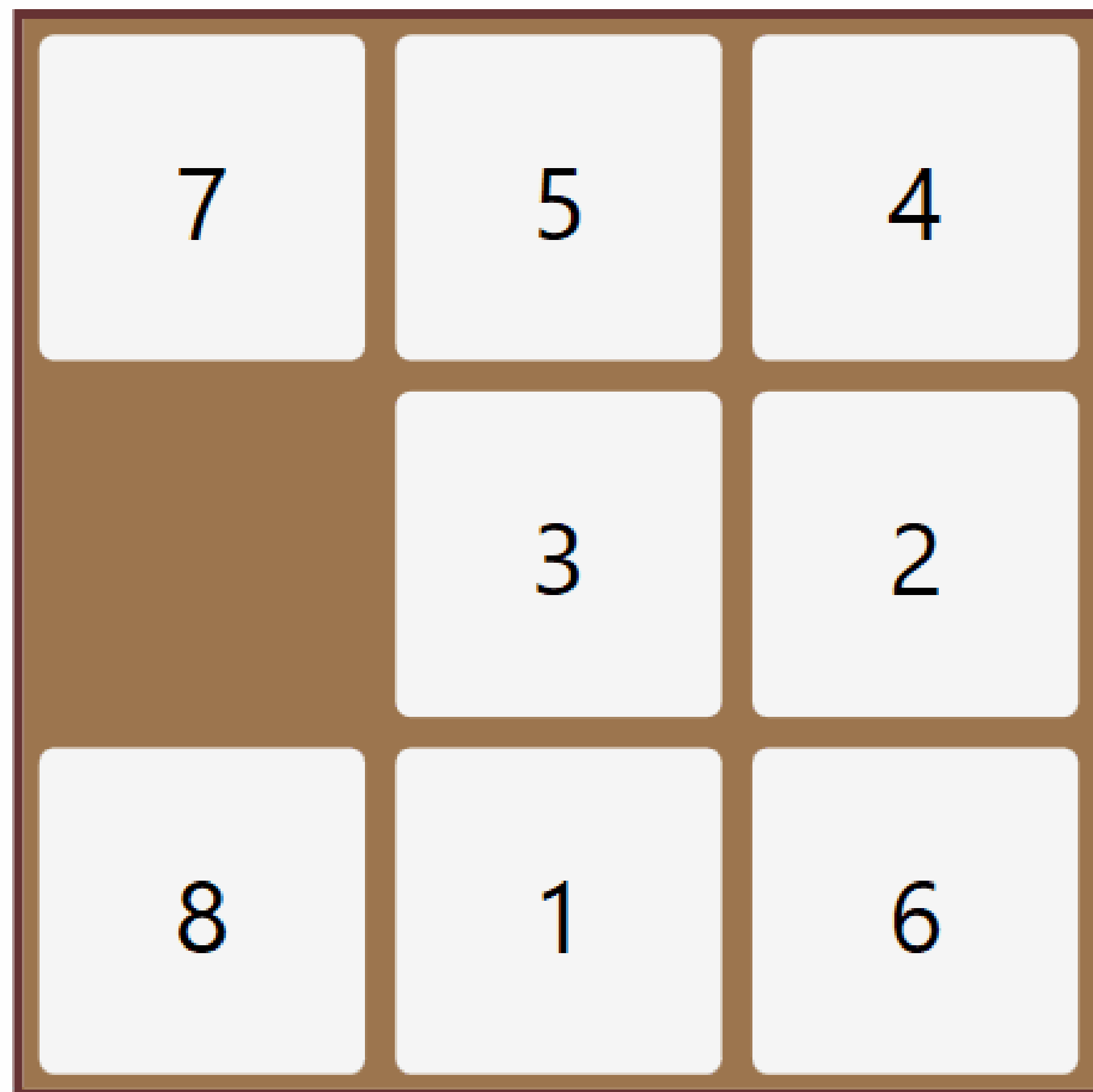
the state in which the agent begins, typically represented as s_i



Key concepts: Actions

Actions

$ACTIONS(s)$: the set of actions, that can be executed in state s .



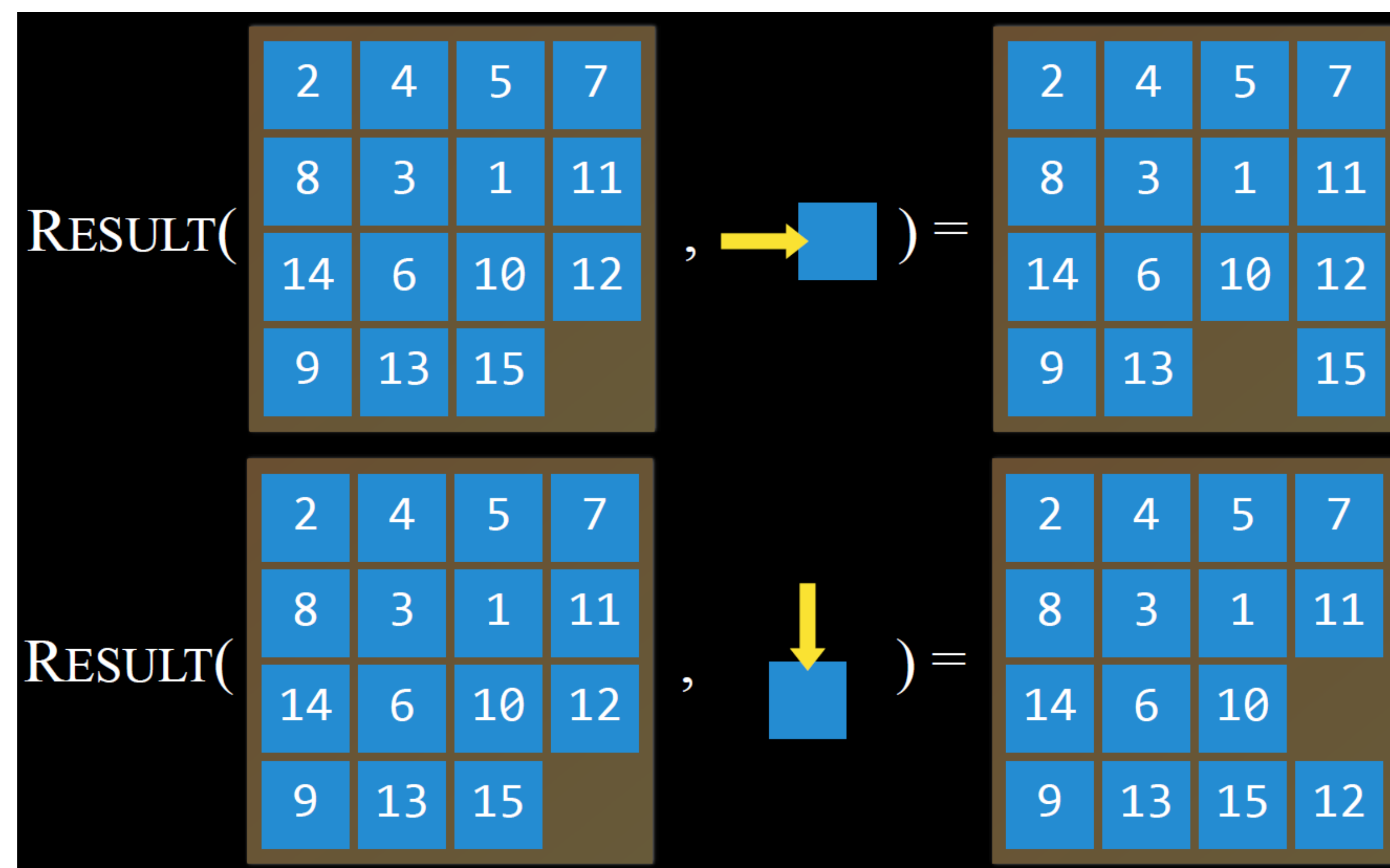
Reference: Stuart Russell & Peter Norvig, *Artificial Intelligence: A Modern Approach* (4th ed.) P83 – P88.

Key concepts: Transition model

Transition model:

maps a state and action to a resulting state

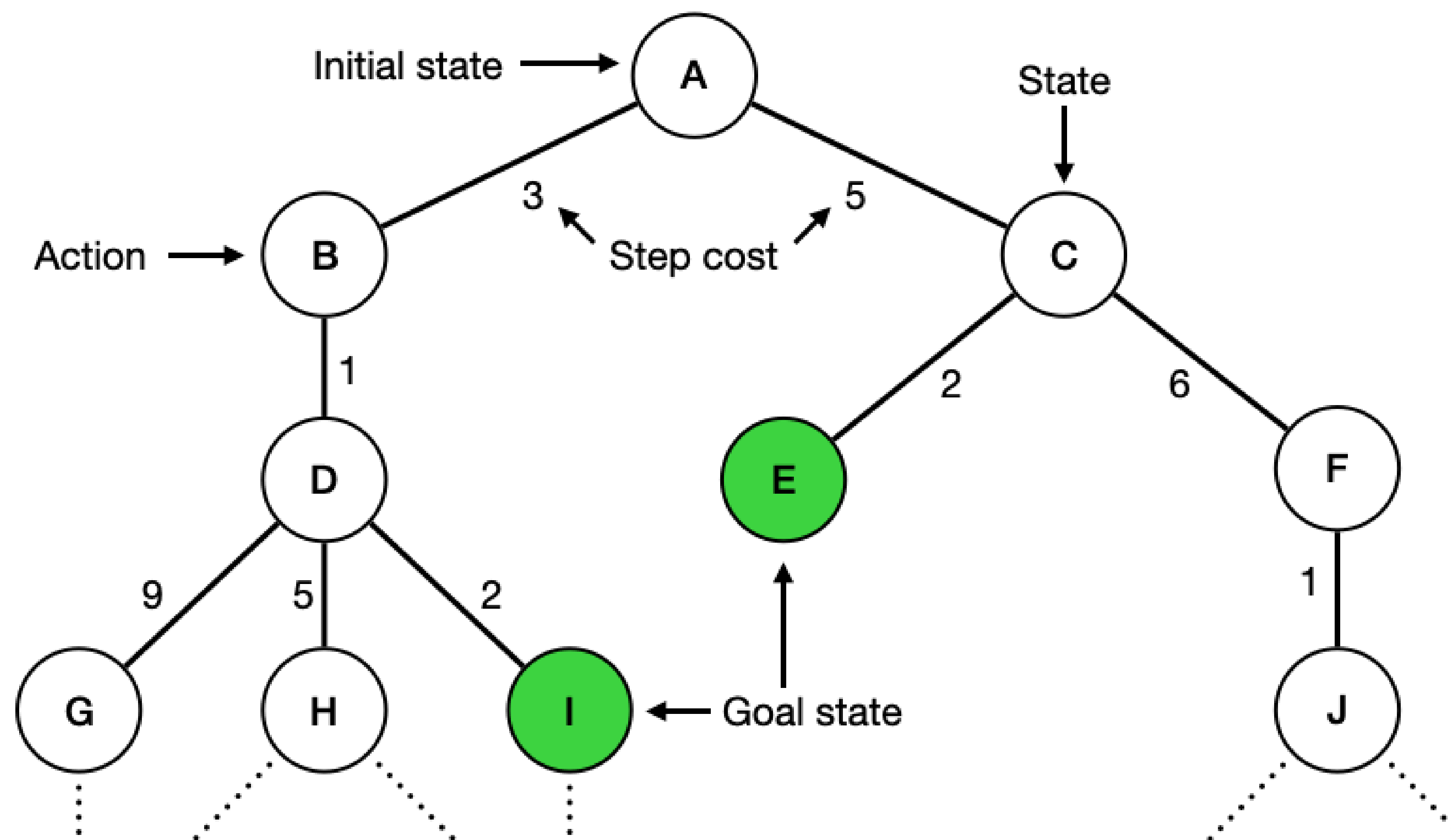
$\text{RESULTS}(s, a)$ returns the state s' , reached after performing action a in state s



Reference: <https://pll.harvard.edu/course/cs50s-introduction-artificial-intelligence-python>

Key concepts: Action cost

Action cost/step cost: $\text{ACTION-COST}(s, a, s')$ or $c(s, a, s')$
the numeric cost of applying action a in state s to reach state s'



Reference: Stuart Russell & Peter Norvig, *Artificial Intelligence: A Modern Approach* (4th ed.) P83 – P88

Search algorithms: Search for a solution

Once the problem is formulated, we need to solve it.

A **solution** is a sequence of *actions* from the *initial state* to a *goal state*.

Optimal solution: A solution is *optimal* if no solution has a lower *path cost*.

Reference: Stuart Russell & Peter Norvig, *Artificial Intelligence: A Modern Approach* (4th ed.) P89 – P91.

Two fundamental search strategies

Tree Search

- Explores the state space as a tree
- Treats each occurrence of a state via a different path as a distinct node
- May generate redundant states

Graph Search

- Explores the state space as a graph
- Tracks visited states to avoid revisiting
- Prevents redundancy and loops

Both are *search strategies*, and specific *search algorithms* are designed based on these strategies.

Reference: <https://www.youtube.com/watch?v=Y1VywhuRFIs>

Basic search strategy I: Tree search

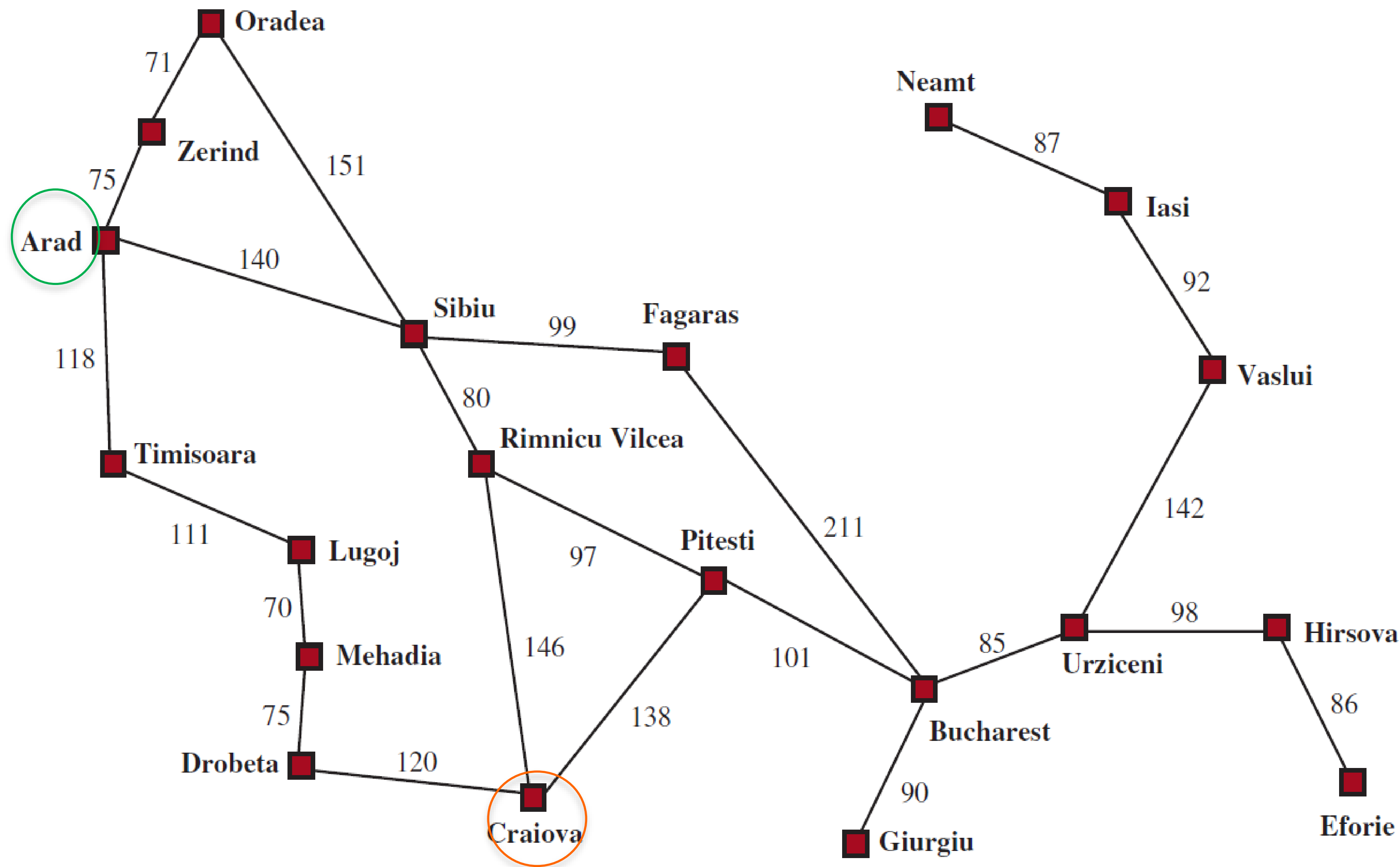
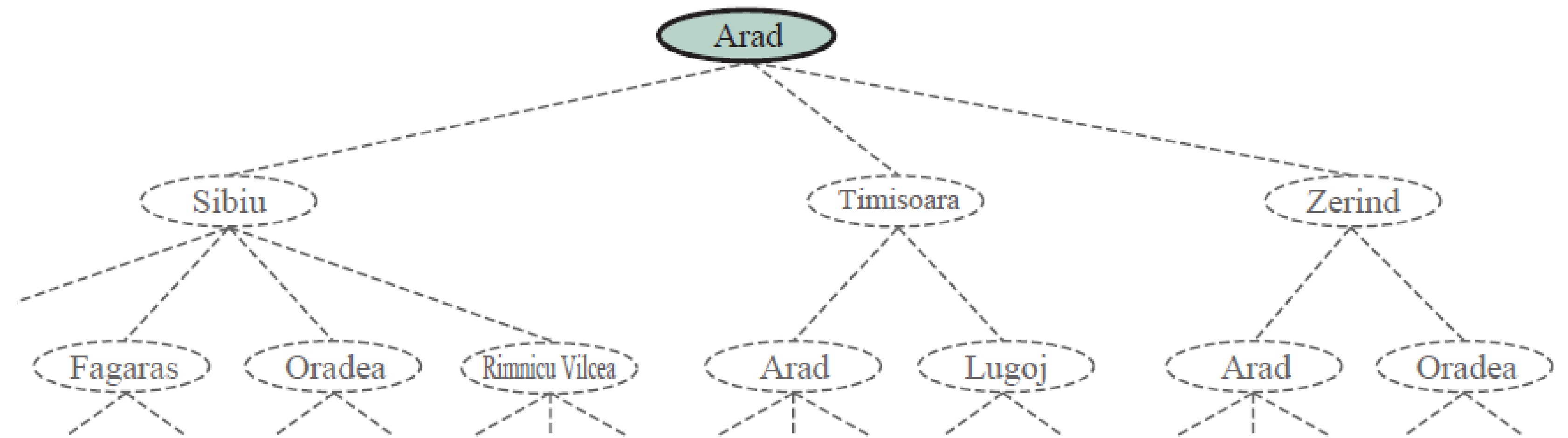
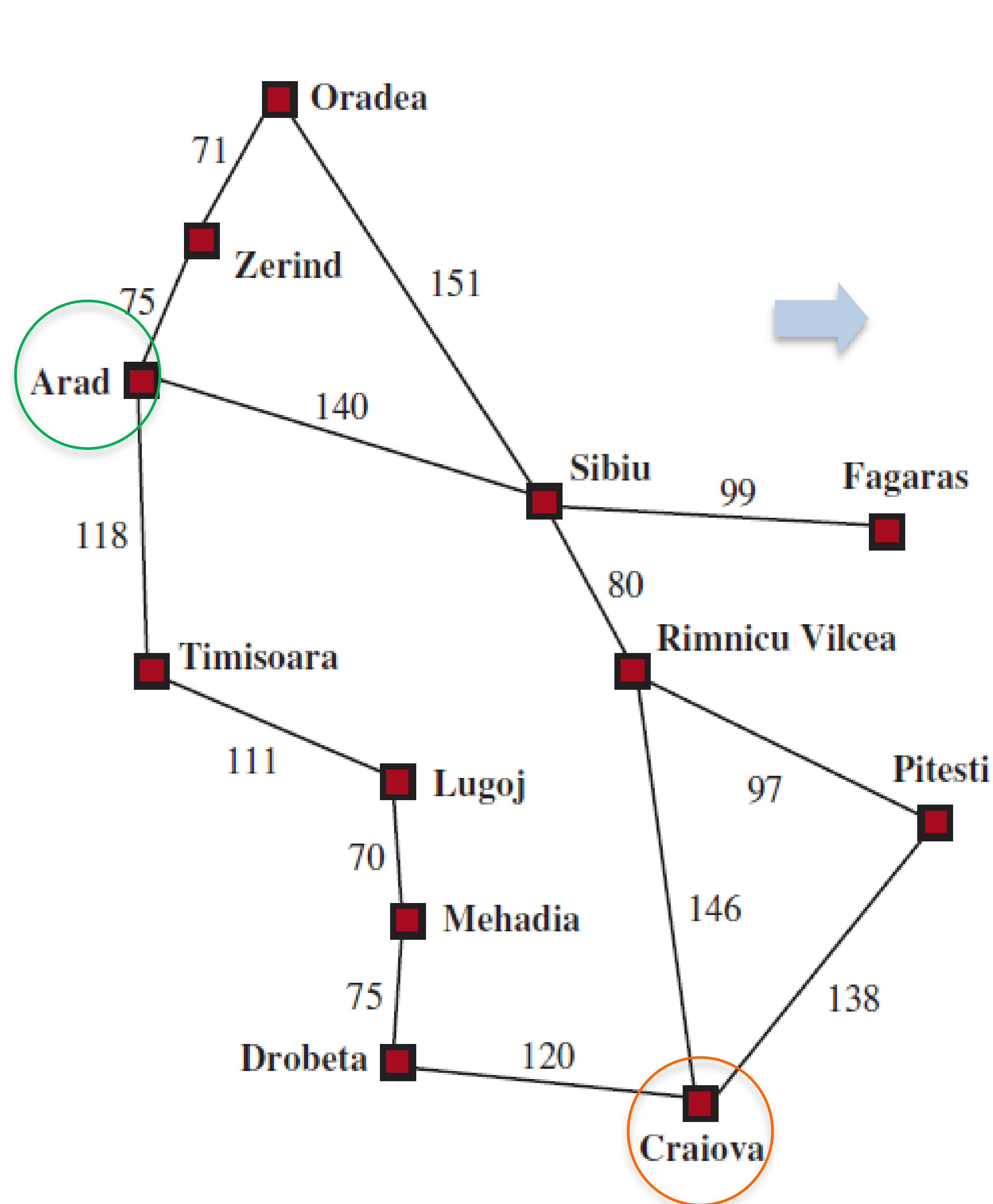


Image source: Stuart Russell & Peter Norvig, *Artificial Intelligence: A Modern Approach* (4th ed.) P82

Basic search strategy I: Tree search



A search tree: A conceptual tree representing possible plans and their outcomes---essentially a "what if" structure.

- The start state is the **root node**
- At each step:
- Check if the current node represents the goal state (**goal test**).
 - If not, "**expand**" the node by applying all **legal actions** to the current state, generating new successor states (child nodes).

Basic search strategy I: Tree search

Tree Search:

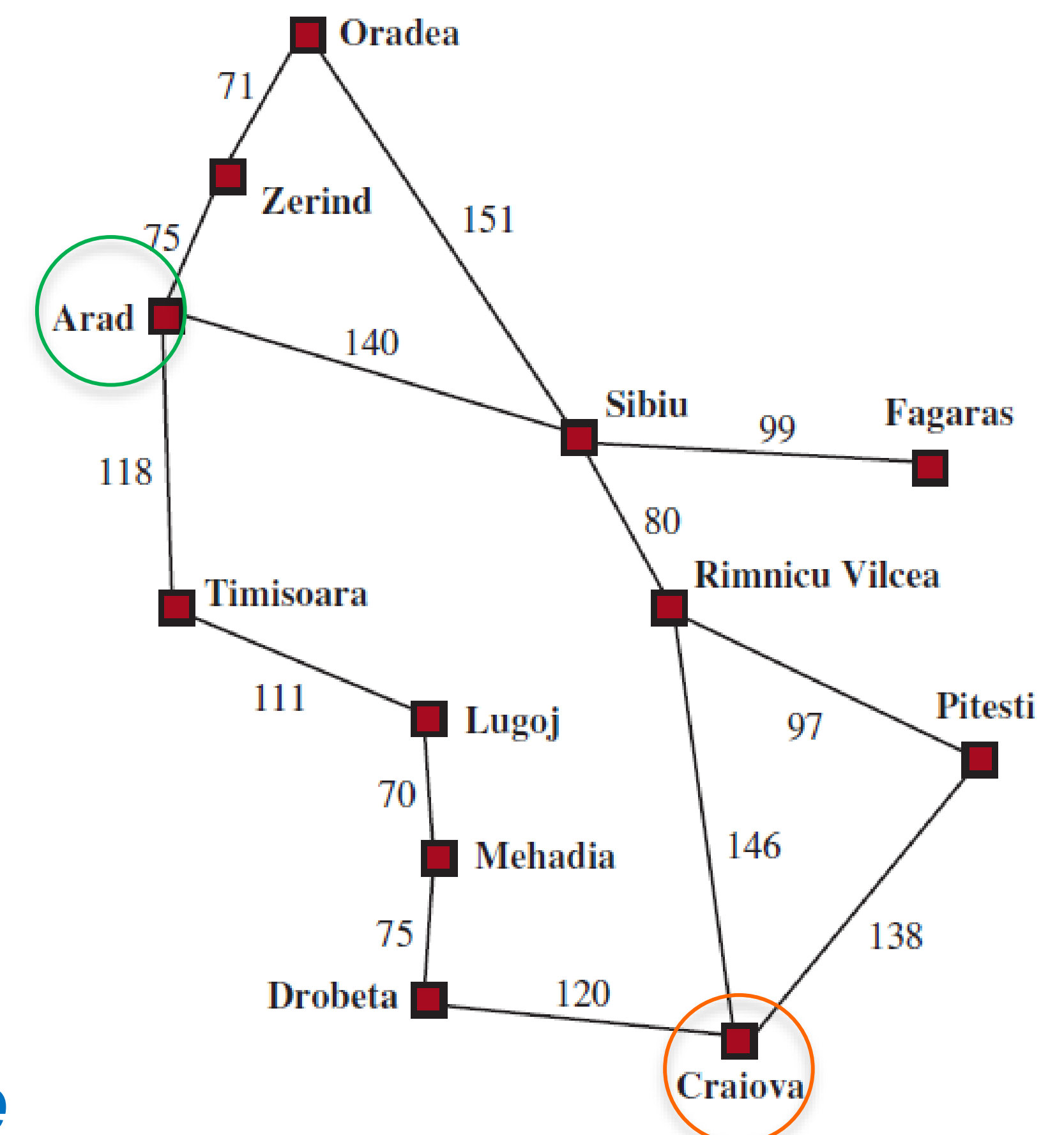
- Enumerates all possible paths from the initial state
- Explores states by generating an explicit search tree

Search Tree Components:

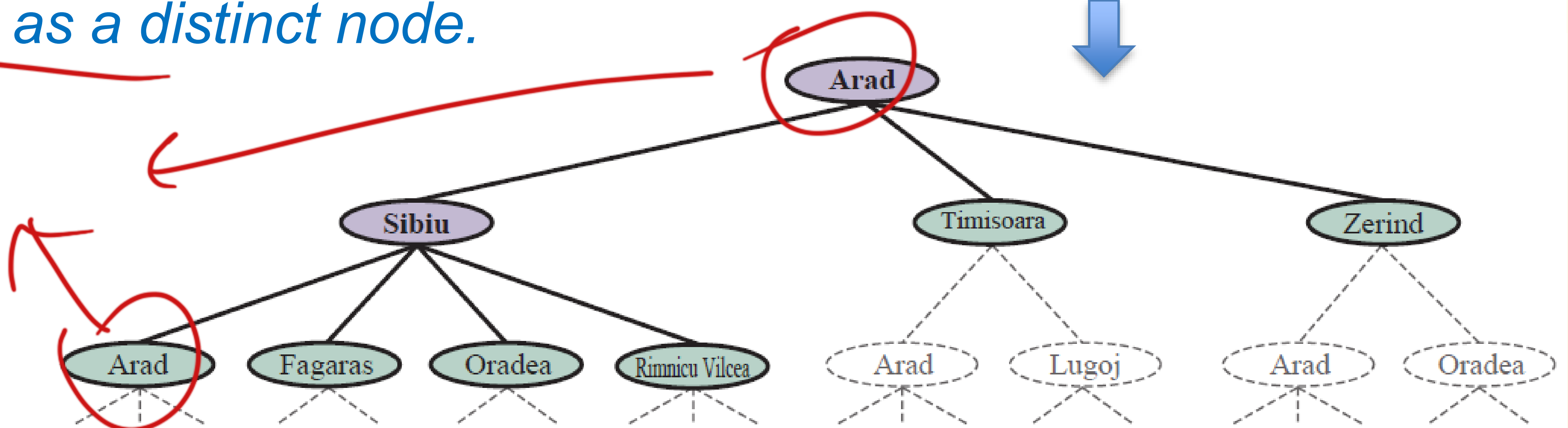
- **Root:** Initial state
- **Nodes:** States generated via the transition model
- **Edges:** Actions connecting states

Key Point:

- *Tree search treats each occurrence of the same state along different paths as a distinct node.*



虽然是在不同路径的 node,
但是是 distinct path



Tree search algorithm: Nodes and expansion

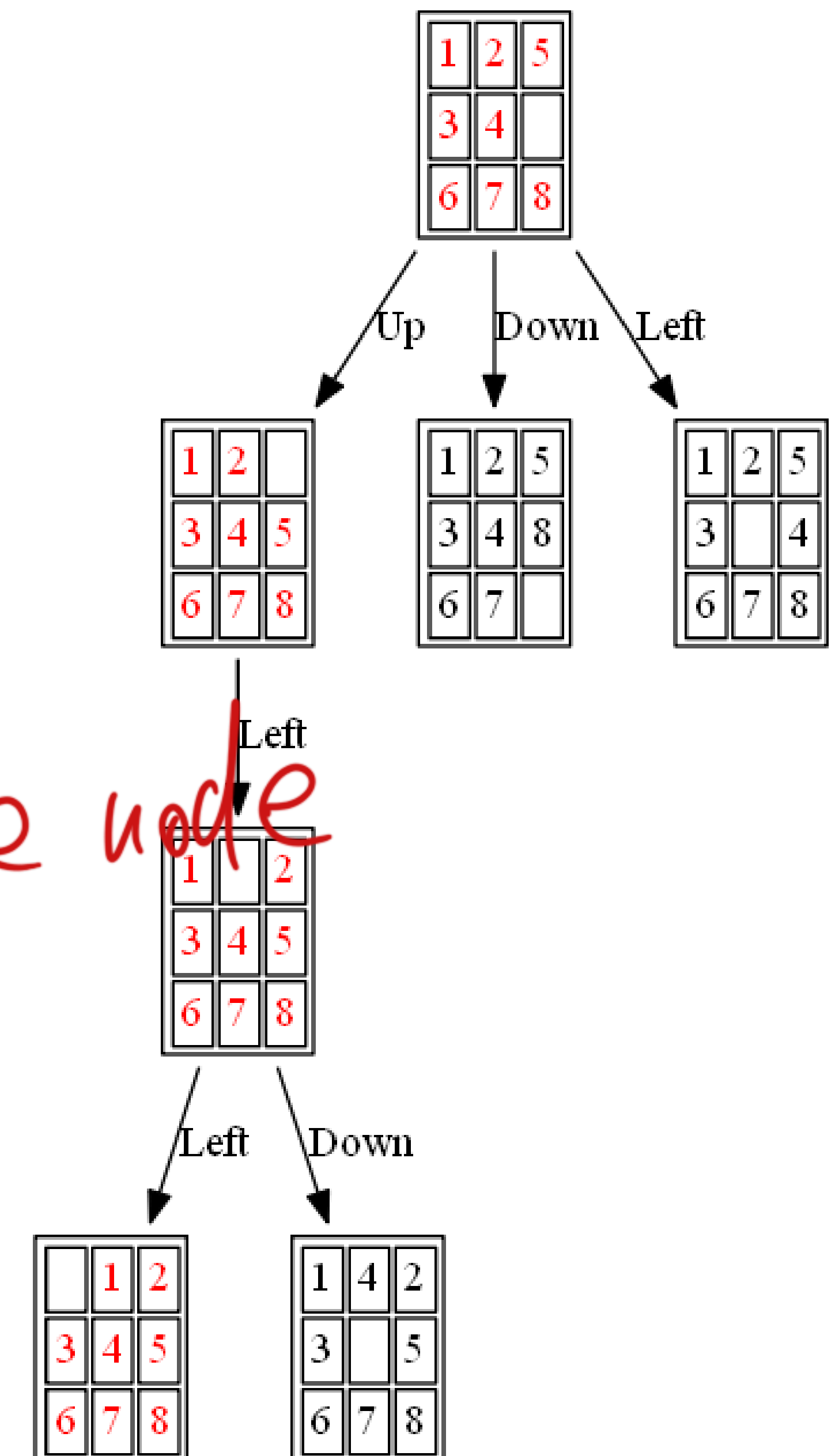
- **Defining a search node:**

- Each node contains a **state**
- Node may also contains additional information:
 - * the **parent node** (which generated this node)
 - * an **action** (action applied to parent to get this node)
 - * **path cost** (cost of the path from initial state to the node)

} things to store in the node

- **Expanding a node:**

- Applying all legal actions to the state contained in the node
- Generating new nodes for all the corresponding successor states



Tree search algorithm: Node management

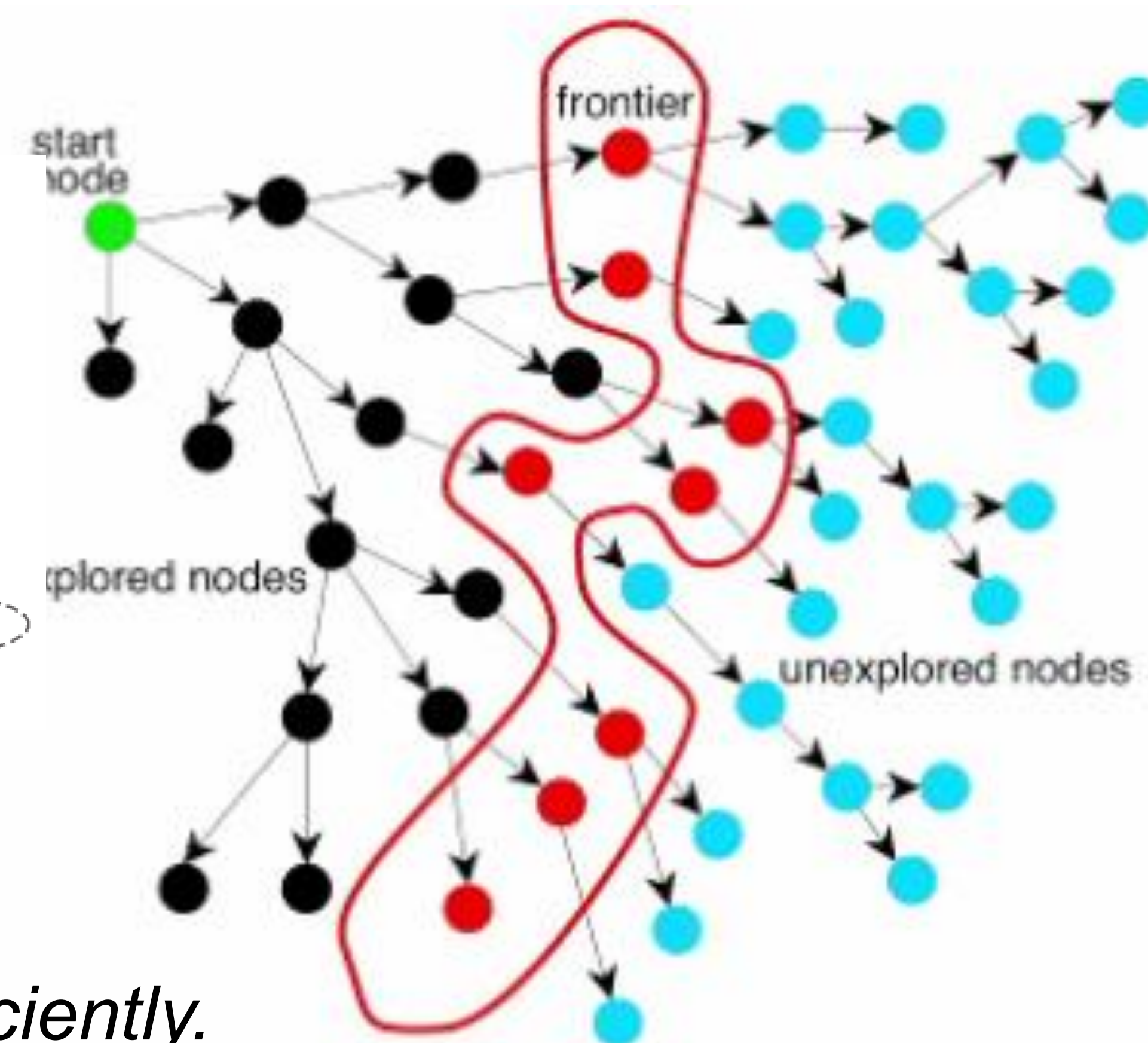
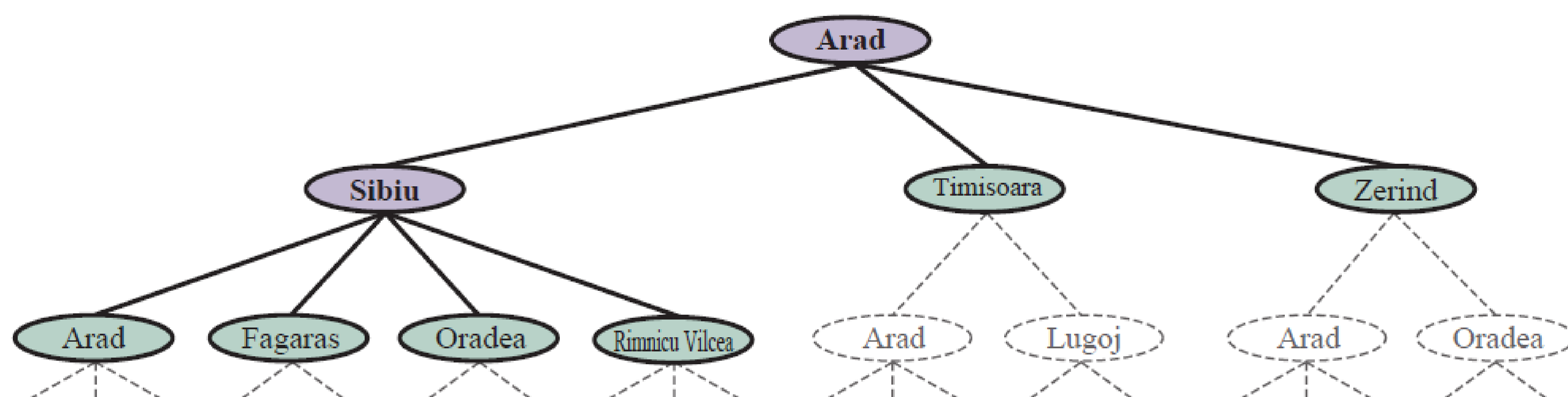
Managing generated nodes

- Need: A data structure* to store nodes as they are generated

Frontier

- Queue (or other structure) of nodes **waiting to be expanded**
- **Stores state and path information**, needed to reconstruct the solution once the goal is reached

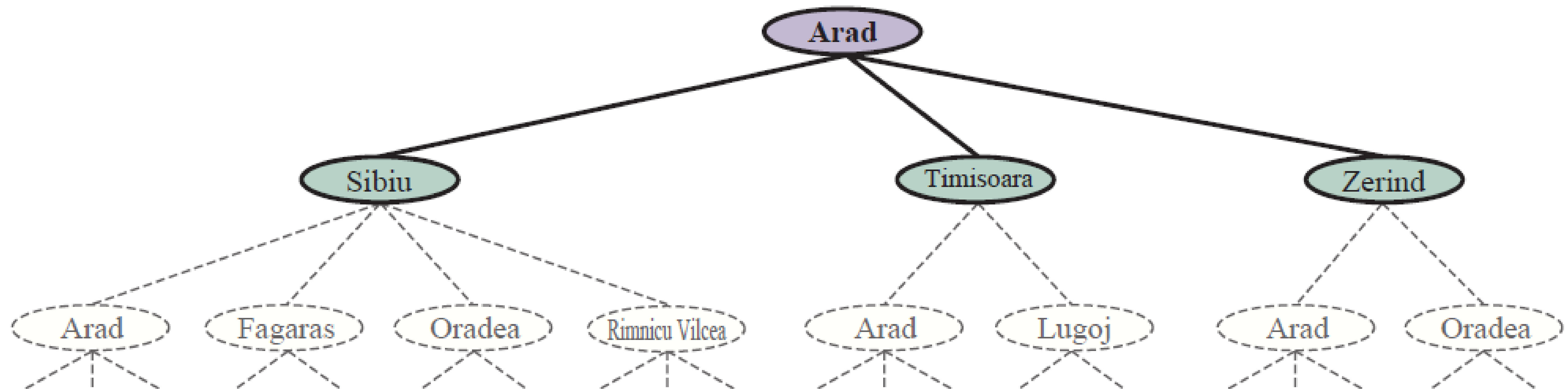
I have seen them
but I haven't expand
them yet.



* A data structure is a way to organize and store data in a computer so it can be accessed, updated, or searched efficiently.

Reference: <https://www.youtube.com/watch?v=xoqcnWXTXcl>

General tree search algorithm

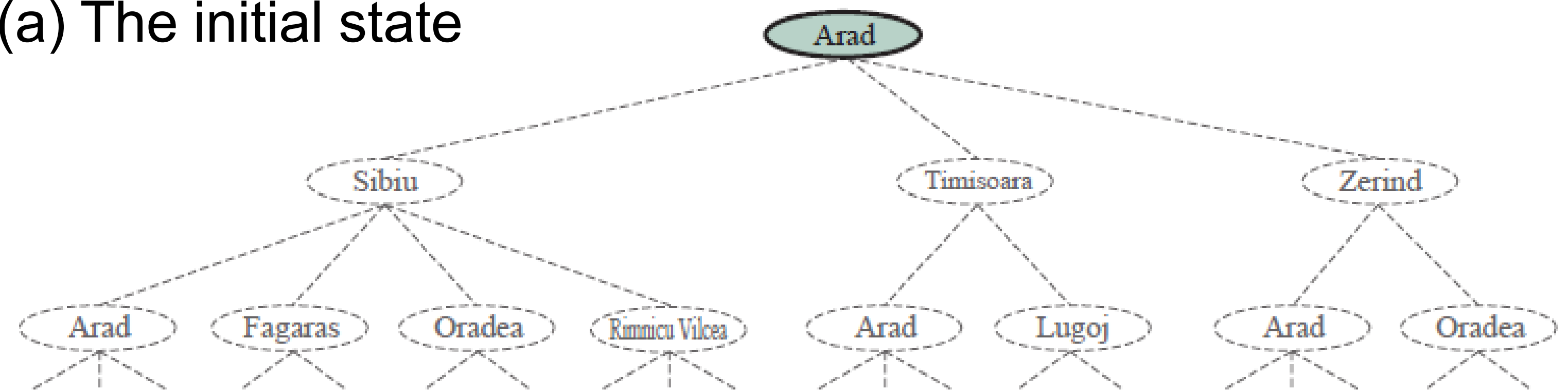


```
function TREE-SEARCH (problem, strategy) return a solution, or failure
  Initialize frontier to the initial state of the problem
  loop do
    if the frontier is empty then return failure
    choose a node from the frontier for expansion according to strategy,
    and remove it from frontier
    if the node contains goal state then return the corresponding solution
    else expand the node, and add the resulting child nodes to the frontier
  end
```

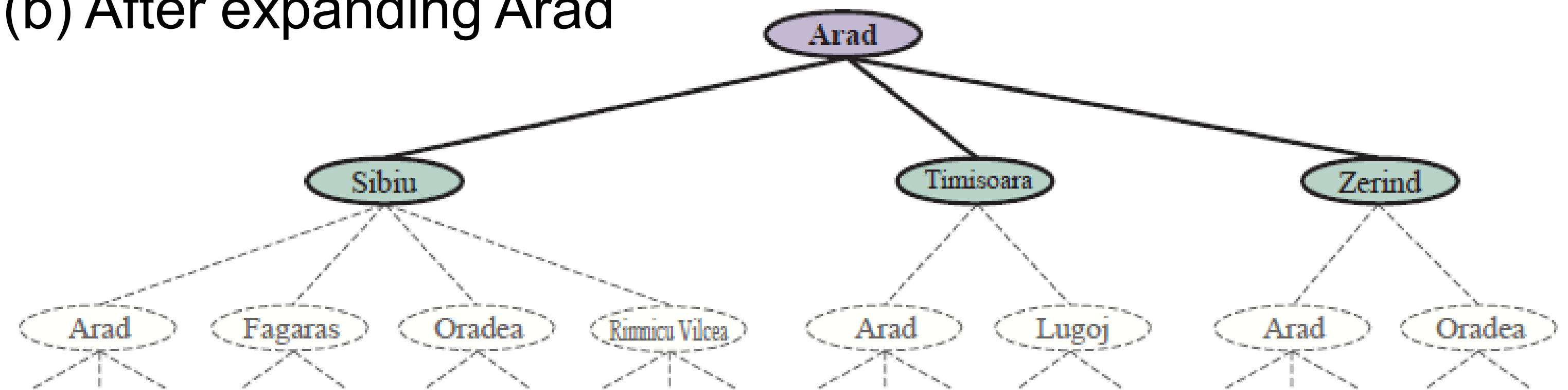
The strategy
determines
search process!

Problem of tree search: Repeated states

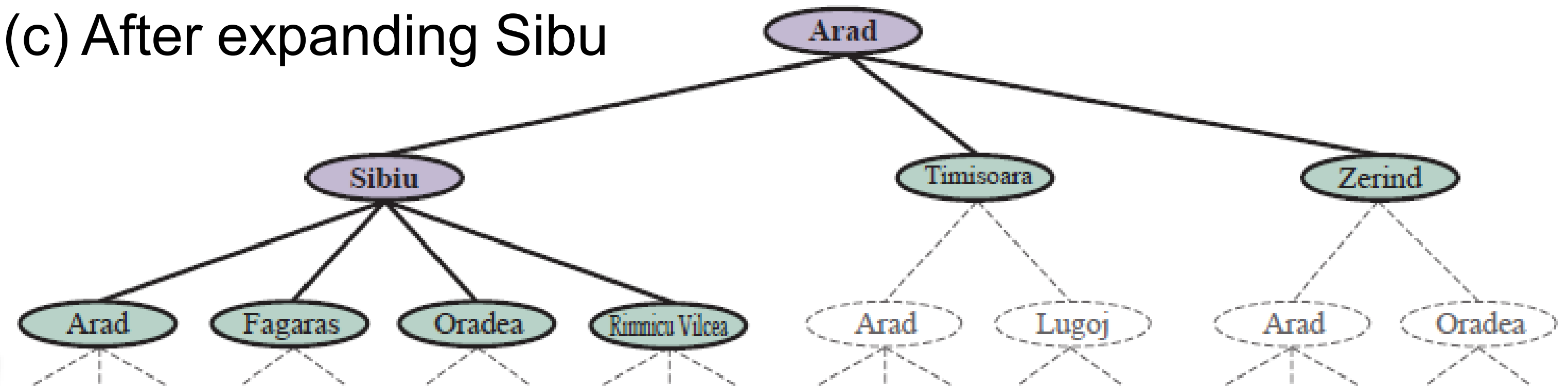
(a) The initial state



(b) After expanding Arad

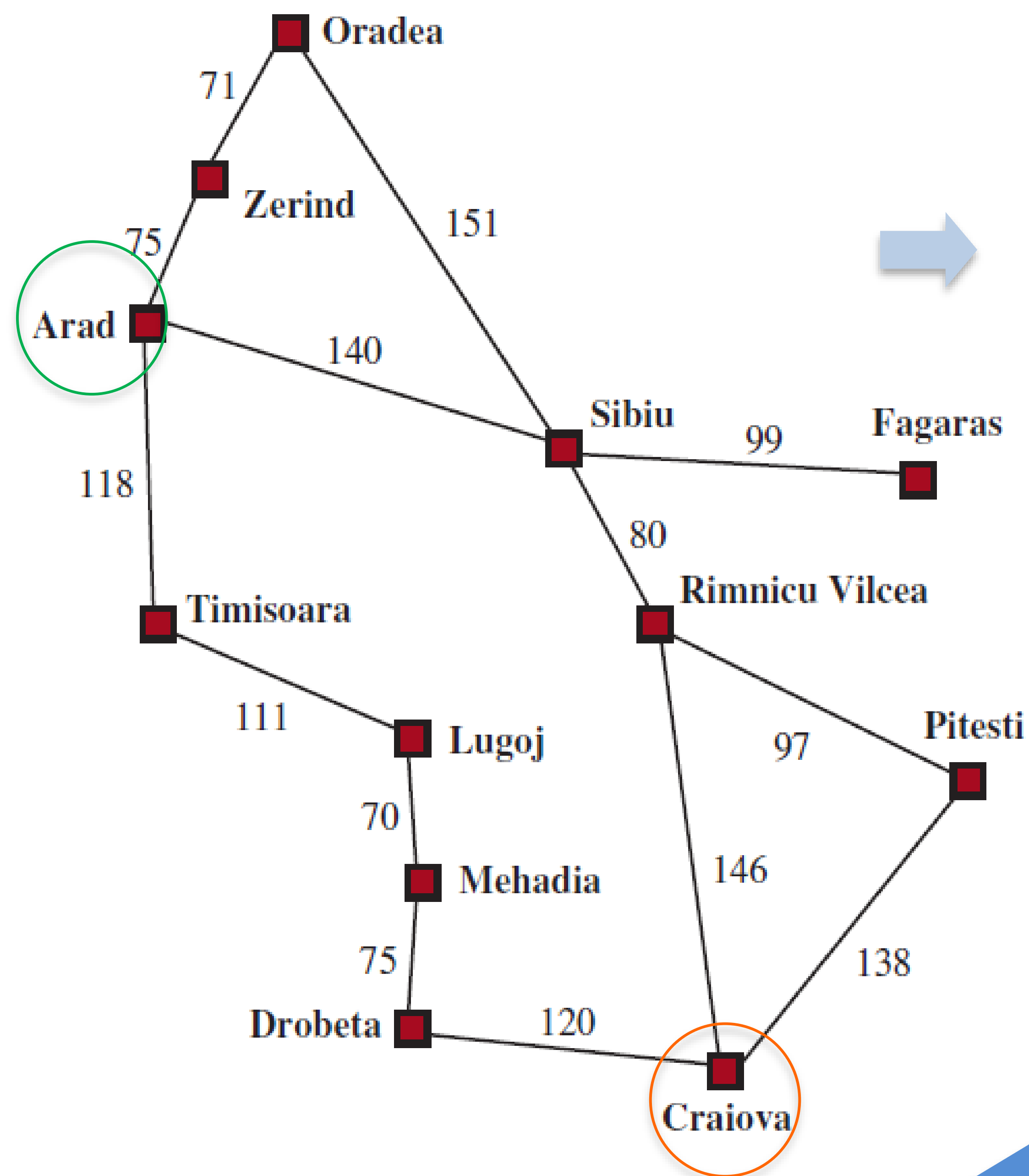


(c) After expanding Sibiu



Arad appears again!

Loops cause redundancy because
path costs are additive

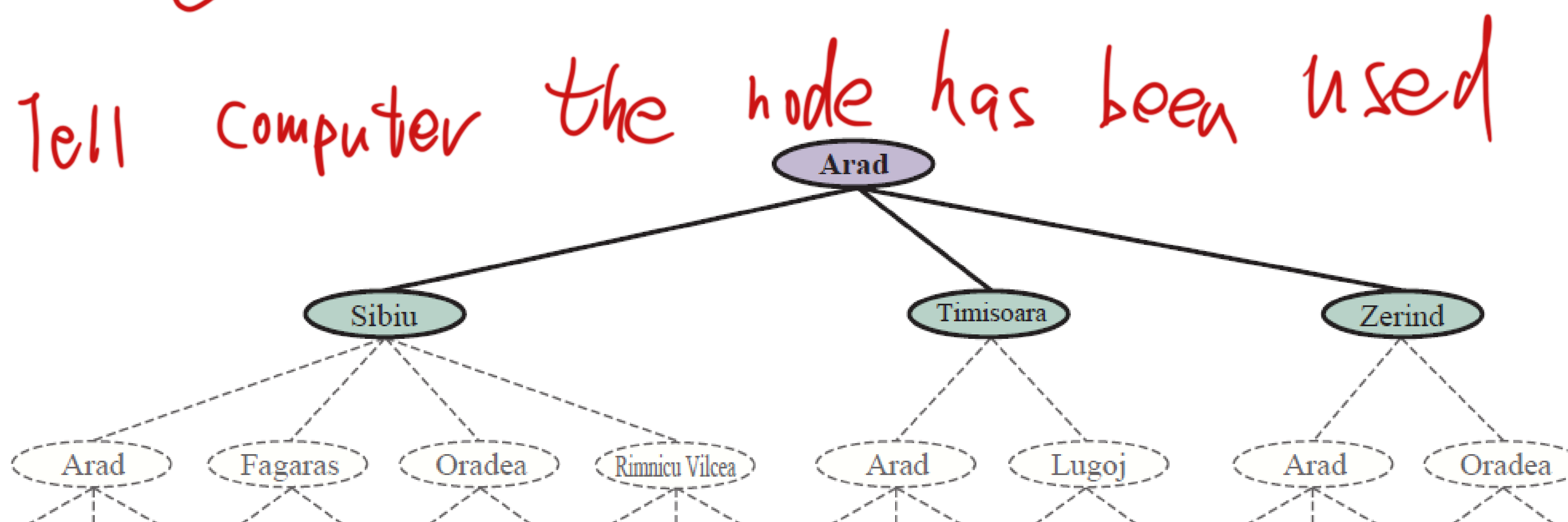


Solution to repeated states

To avoid redundancy and inefficiency caused by revisiting the same states, we augment the basic tree search with one additional data structure:

Explored set: A record of all previously visited nodes (states).

Helps prevent revisiting the same state, reducing loops and redundancy.



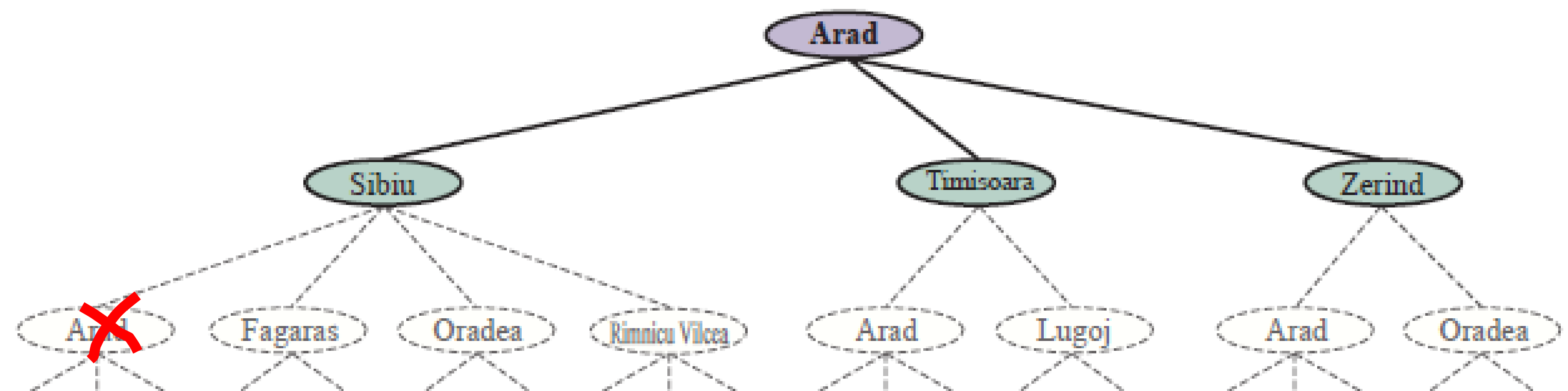
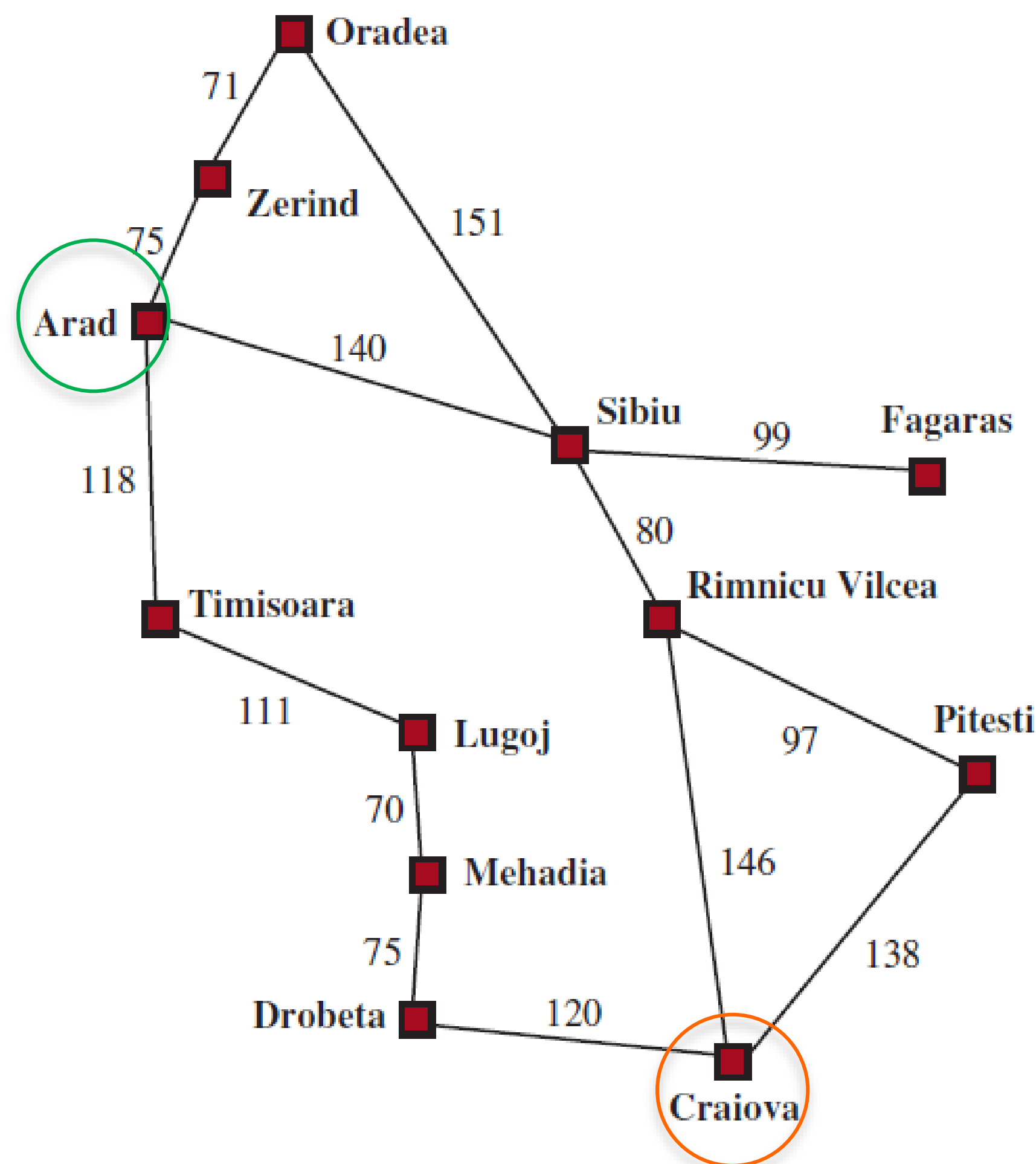
Lavender: Nodes that have been expanded (in the **explored set**)
Green: nodes currently on the **frontier**, waiting to be expanded.

Image source: Stuart Russell & Peter Norvig, *Artificial Intelligence: A Modern Approach* (4th ed.) P89

Basic search strategy II: Graph search

Graph Search: Simple modification of tree search

- Before adding a node to the frontier, check if its state has been visited
- Requires keeping track of all visited states (can use significant memory)
e.g.: 8-puzzle has $9!/2 \approx 182,000$ possible states
- Although we are exploring a graph, the *expansion process still forms a tree-like structure*



Reference: <https://www.youtube.com/watch?v=Y1VywhuRFIs>

Graph search vs Tree search

Tree search

```
function TREE-SEARCH(problem, strategy) return a solution, or failure
  Initialize frontier to the initial state of the problem
  loop do
    if the frontier is empty then return failure
    choose a node from the frontier for expansion according to strategy,
    and remove it from frontier
    if the node contains goal state then return the corresponding solution
    else expand the node, and add the resulting child nodes to the frontier
  end
```

Graph search

```
function GRAPH-SEARCH(problem, strategy) return a solution, or failure
  Initialize frontier to the initial state of the problem
  Initialize the explored set to be empty
  loop do
    if the frontier is empty then return failure
    choose a node from the frontier for expansion according to strategy,
    and remove it from frontier
    if the node contains goal state then return the corresponding solution
    add the node to the explored set
    expand the node, and add the resulting nodes to the frontier
    only if not in the frontier or explored set
  end
```

The strategy leads to a variety of search algorithms!

Newly generated nodes that match the frontier nodes or explored nodes are discarded.

Reference: <https://www.youtube.com/watch?v=Y1VywhuRFIs>

Summary: Tree search vs graph search



Aspect	Tree search	Graph search
Explored set	No (no memory of visited states)	Yes (maintains a set of visited states)
Memory use	Low	High (stores all visited states)
Time efficiency	Low (may revisit states many times)	High (avoids revisiting)
Risk of infinite loops	High (esp. in graphs with cycles)	Low (prevents loops via state tracking)
Implementation complexity	Simpler	More complex
When to use	- Memory-constraint systems - Cycles are impossible or negligible.	- Cyclic or redundant state spaces - Sufficient memory is available
Typical use cases	Scenarios with huge state spaces and no cycles, memory-limited systems	Most real-world problems with state reuse (e.g., pathfinding, games and puzzles)

Node exploration strategies:

Uninformed vs informed search

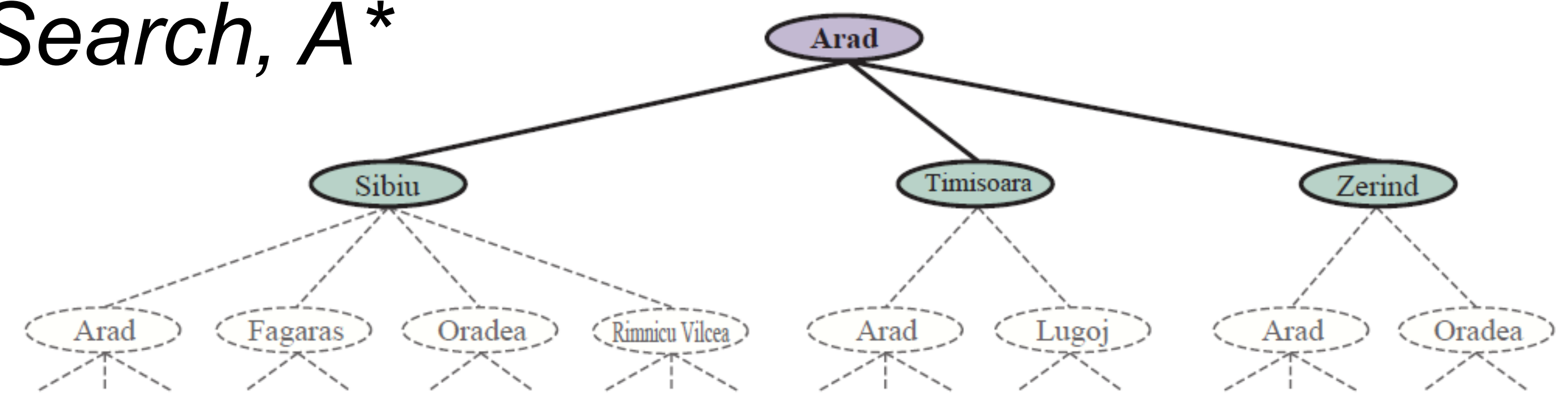
The **node exploration strategy** leads to a variety of search algorithms.

Uninformed search:

- Explores the search space without additional knowledge (e.g., the goal's location).
- Examples: *Breadth-First Search (BFS)*, *Depth-First Search (DFS)*, *Uniform-Cost Search/Dijkstra's algorithm*

Informed search:

- Uses domain knowledge (*heuristics*, e.g., an estimate of the distance to the goal) to guide the search towards the goal.
- Examples: *Greedy Best-First Search*, A^*



Evaluation of search algorithms

We can evaluate an algorithm's performance in four ways:

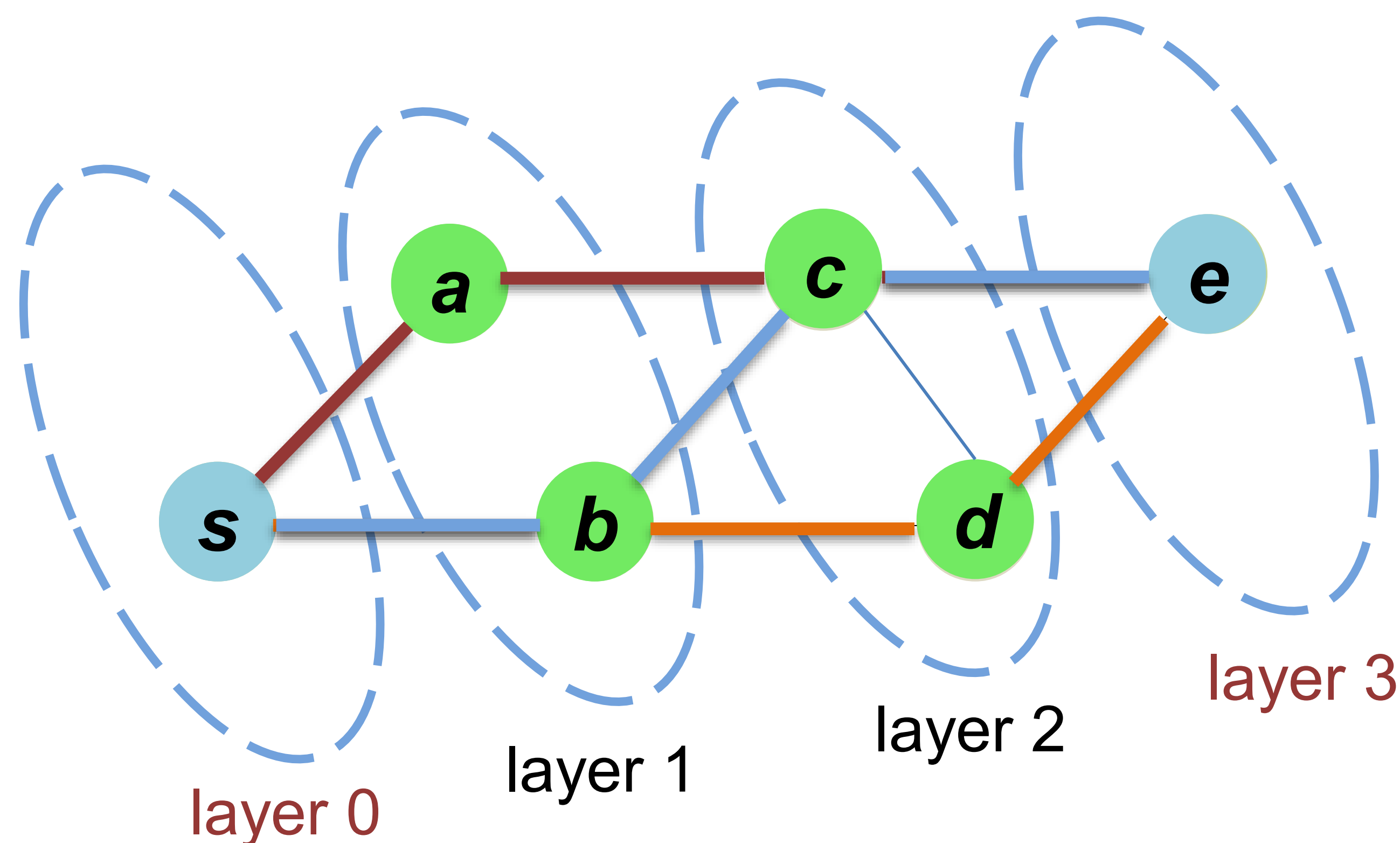
- **Completeness:** Does the algorithm always find a solution if one exists?
- **Optimality:** Does it find the lowest-cost (best) solution among all possible ones?
- **Time complexity:** How long does it take to find a solution?
- **Space complexity:** How much memory is needed to perform the search?

These criteria apply to evaluating *general problem-solving algorithms*.
In our course, we will focus on *completeness* and *optimality*.

Reference: Stuart Russell & Peter Norvig, *Artificial Intelligence: A Modern Approach* (4th ed.) P93.

Breadth-first search (BFS)

- Explore nodes in “layers” (level by level)
- Can find everything reachable from a given starting node
- Can compute minimum hops between two nodes (i.e., shortest path, if all steps cost the same)



$s \rightarrow e$: minimum 3 hops

$s \rightarrow a \rightarrow c \rightarrow e$

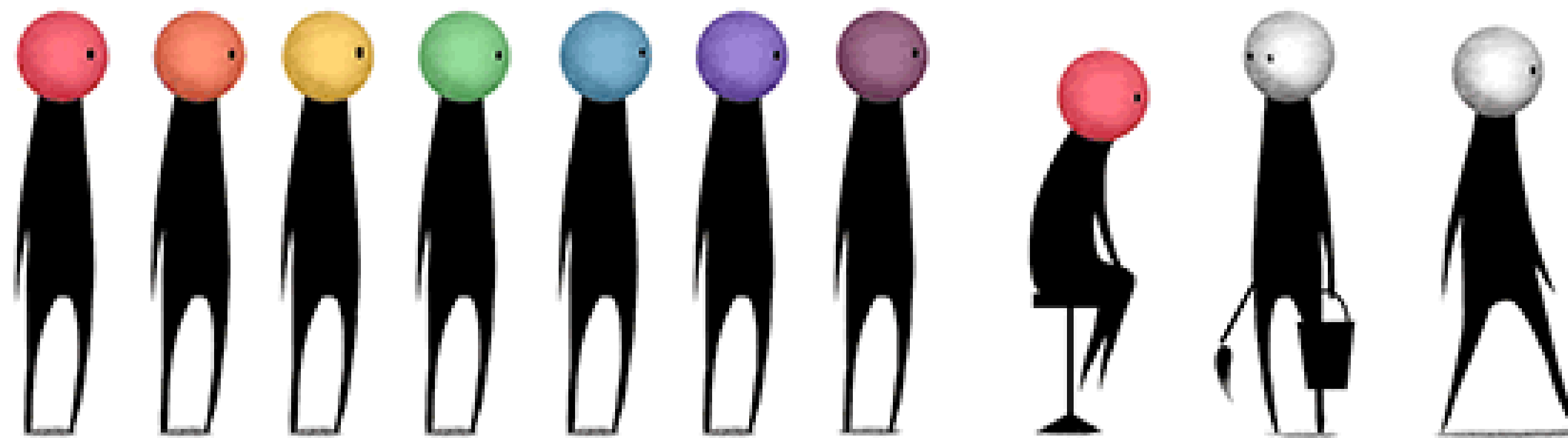
$s \rightarrow b \rightarrow d \rightarrow e$

$s \rightarrow b \rightarrow c \rightarrow e$

BFS: Frontier as a FIFO queue

- BFS always expands the *shallowest node* in the frontier
- In practice, the **frontier** in BFS is implemented as a **queue** (FIFO) to ensure nodes are expanded in the order they are discovered

Frontier: A queue (or other data structure) of nodes waiting to be expanded.



Queue: A First-In-First-Out (FIFO) data structure, where the first element added is the first to be removed.

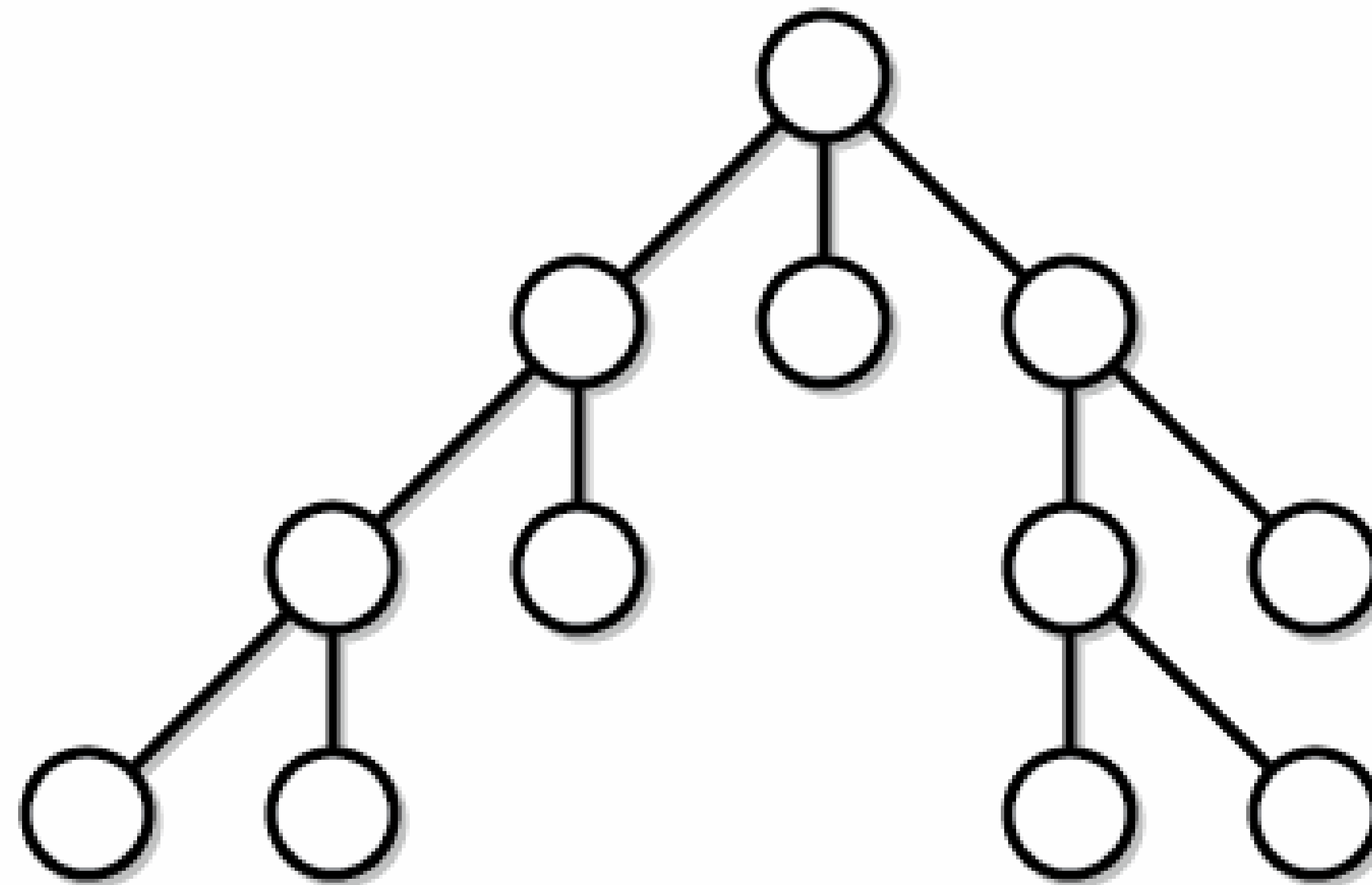
Reference: Stuart Russell & Peter Norvig, *Artificial Intelligence: A Modern Approach* (4th ed.) P94 – P95.

Depth-first search (DFS)

Strategy: Expand the **deepest** node in the frontier first.

Explore one path as far as it can go before backtracking.

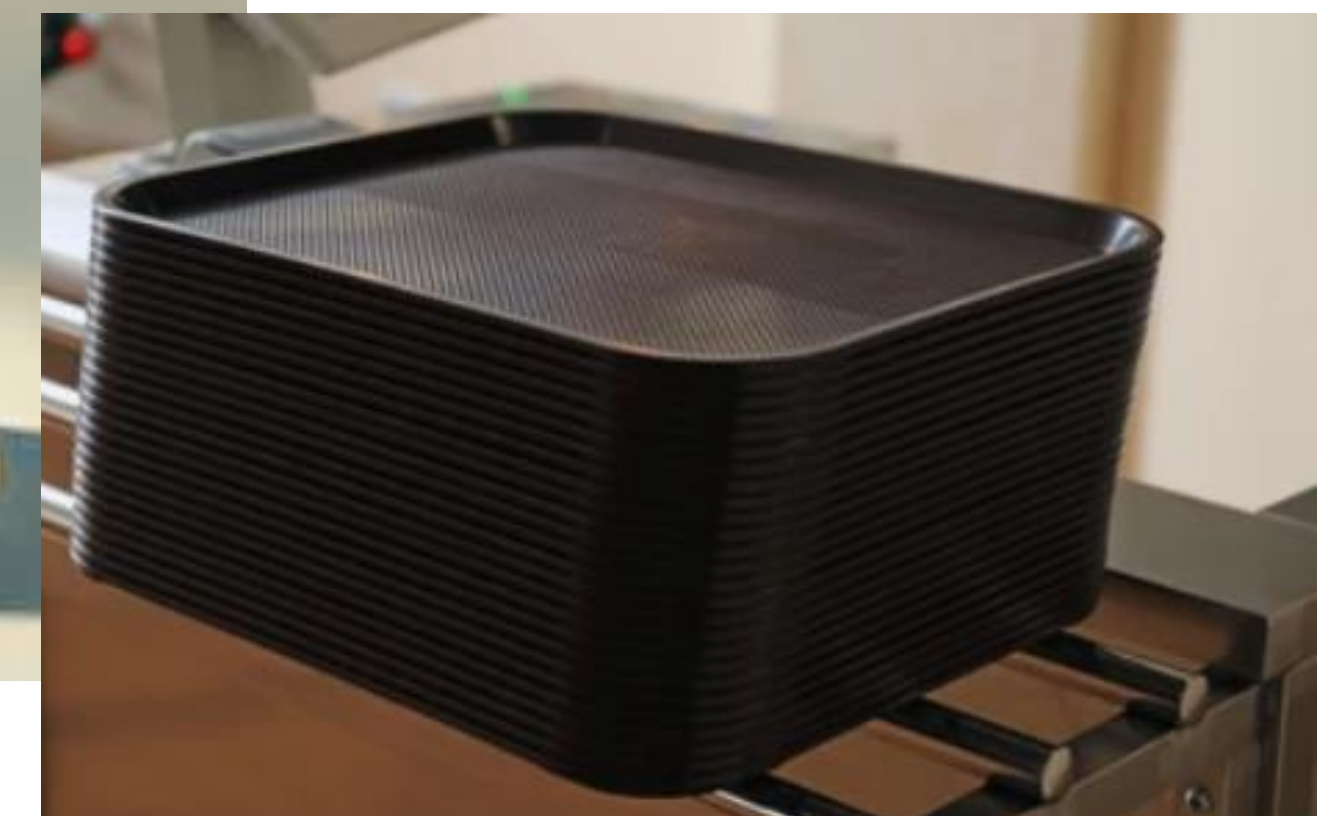
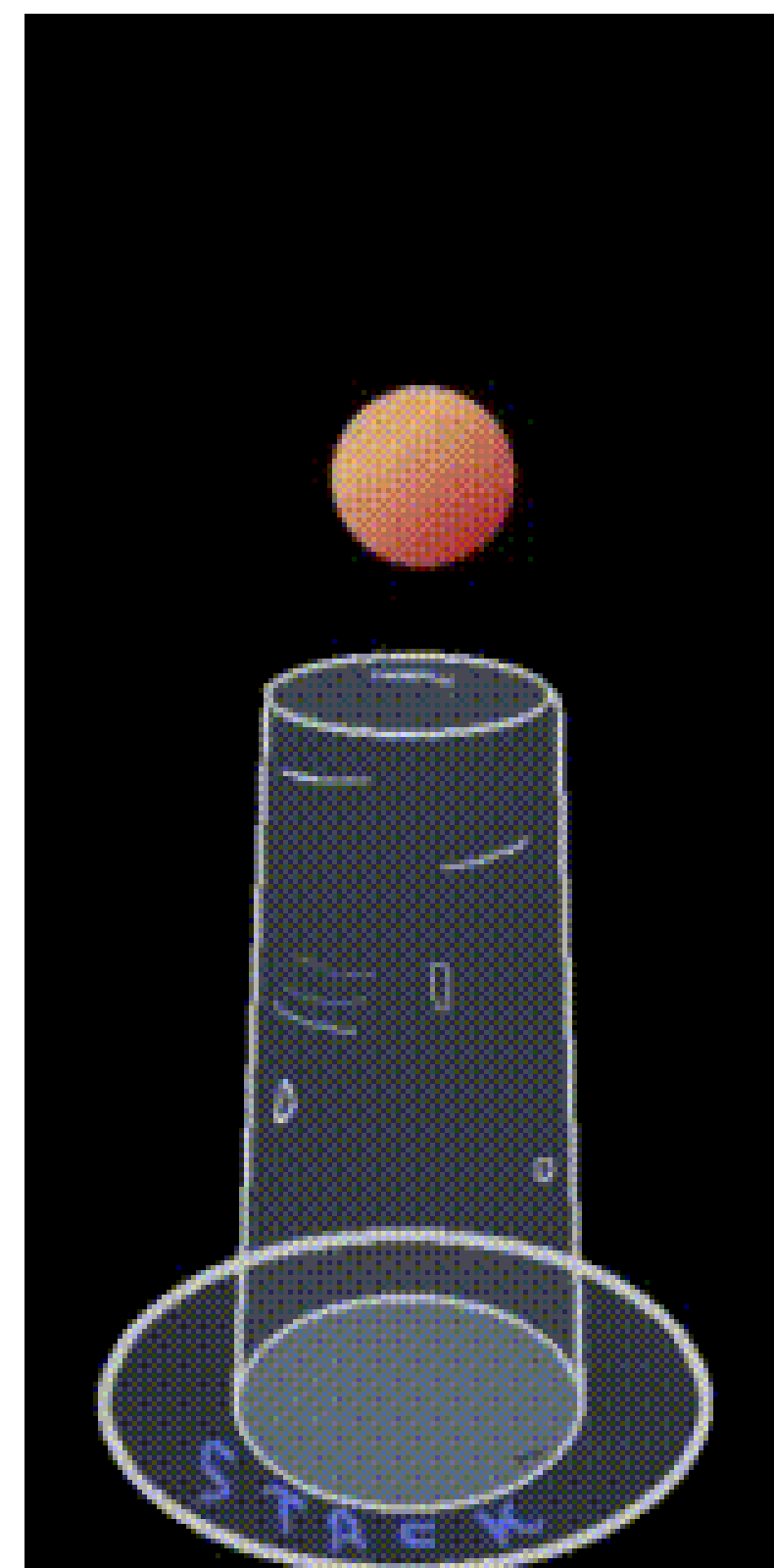
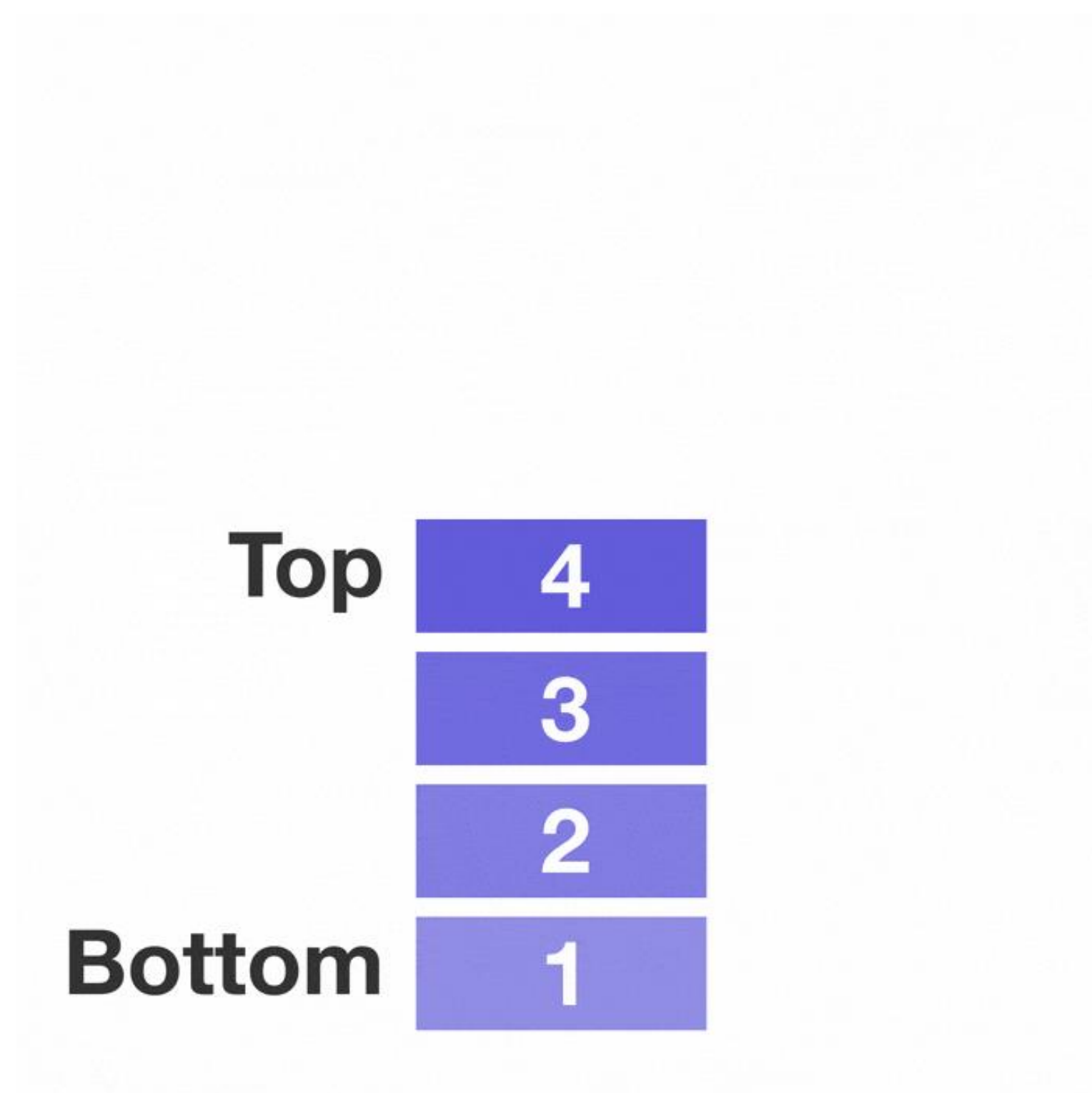
All the successors of a node are expanded **before** moving to other siblings.



Reference: Stuart Russell & Peter Norvig, *Artificial Intelligence: A Modern Approach* (4th ed.) P96 - P99.

DFS: Using a LIFO stack for the Frontier

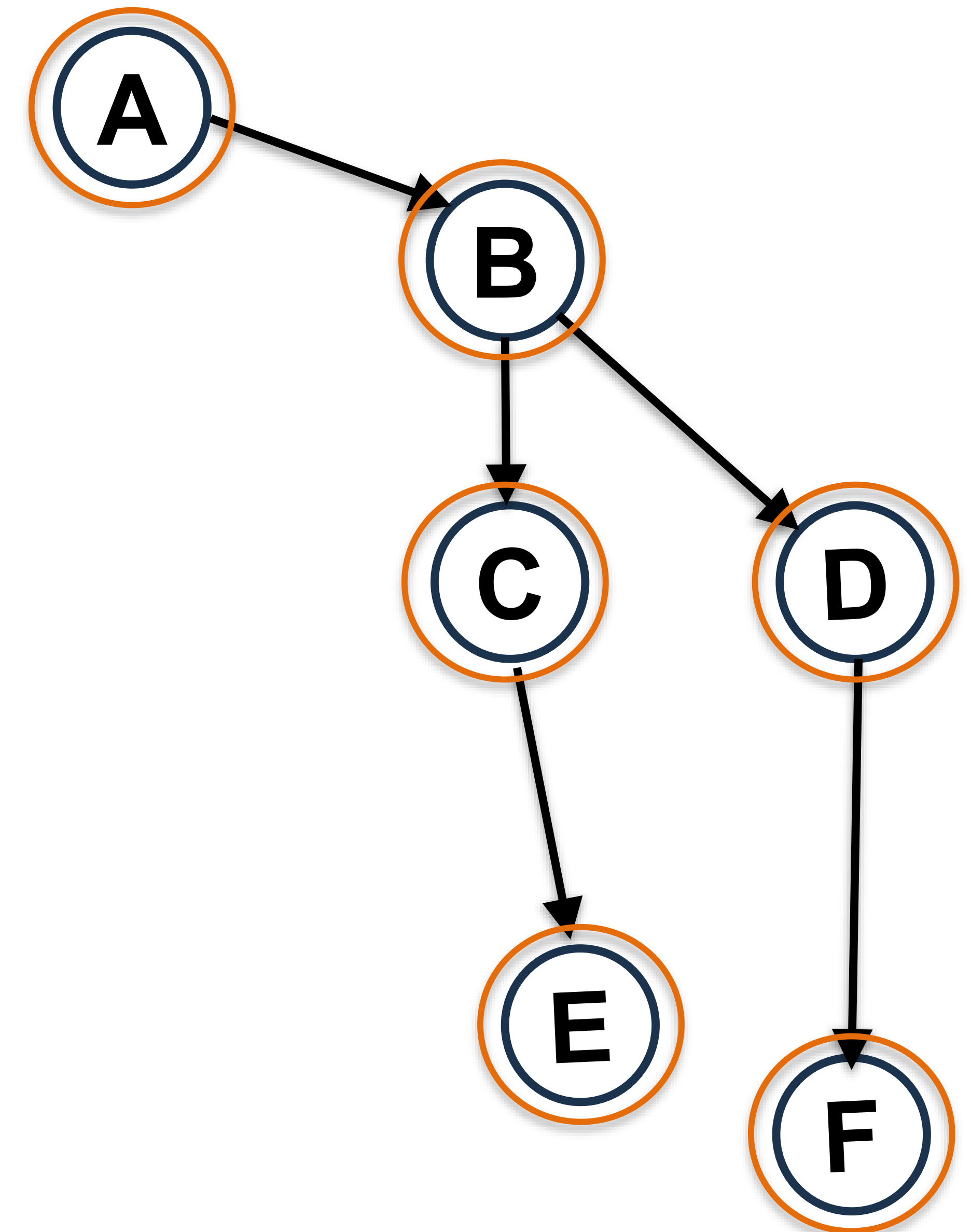
- DFS always expands the *deepest node* in the frontier
- In practice, the **frontier** is implemented as a **stack** (LIFO) to explore one branch fully before backtracking



Stack: A Last-In-First-Out (LIFO) data structure, where the *last* element added is the *first* to be removed.

Finding a path from A to E using DFS

- Explore one path as far as it can go before backtracking.
- All the successors of a node are expanded before moving to other siblings.
- Often used for tasks like puzzle solving or analyzing maze-like structures where a solution path may be deep.



Summary: BFS vs DFS

Feature	BFS (Breadth-First Search)	DFS (Depth-First Search)
Search style	Explores all nearby nodes first (level by level)	Goes as deep as possible before backtracking
Data structure used	Queue (FIFO - First in, first out)	Stack (LIFO - Last in, first out)
Will it always find a solution if one exists? (Completeness)	Yes, if a solution exists	Not guaranteed – may get stuck in deep or infinite paths
Will it find the shortest solution? (Optimality)	Yes, if all steps cost the same	No, may find a longer or less efficient path
Time needed	Slower if the solution is deep	Can be faster if the solution is deep
Memory needed	High – remembers all paths at one level	Low – tracks only one path at a time
Best for	Finding shortest paths (e.g., in maps) Small search space	Exploring puzzles or scenarios with deep solutions Large or deep search spaces
When to use	Need shortest route Equal step cost Enough memory	Memory is limited When solution may be far/deep Don't need shortest path